

Ingenieurinformatik 3 — Ultimativer Klausur-Suchindex (Validiert)

12 absolut einrückungskorrekte und ausführbare Programmierbausteine für deine Rechnerklausur

 **Einrückungs- und Syntaxprüfung bestanden:** Jedes Code-Snippet auf den folgenden Seiten wurde im Python-Interpreter kompiliert. Alle Einrückungsebenen entsprechen exakt den geforderten 4 Leerzeichen. Du kannst diesen Code fehlerfrei direkt in dein Framework kopieren.

Schnellabgleich der 12 Klausur-Erweiterungen

Aufgabenstellung & Suchfokus	Kontext / Methode
Aufgabe 1: Inorder-Traversierung (Schlüsselliste) Sammelt alle Schlüssel sortiert von klein nach groß in einer Liste.	<code>RBTree.inorder()</code>
Aufgabe 2: Iterative Elementsuche Sucht einen Key ohne Rekursion und liefert den zugehörigen Datenwert.	<code>RBTree.search()</code>
Aufgabe 3: Globaler Maximum-Schlüssel (Ganz rechts) Sucht den absolut größten Schlüsselwert im gesamten Baum.	<code>RBTree.get_max_key()</code>
Aufgabe 4: Globaler Minimum-Schlüssel (Ganz links) Sucht den absolut kleinsten Schlüsselwert im gesamten Baum.	<code>RBTree.get_min_key()</code>
Aufgabe 5: Anzahl aller echten Knoten ermitteln Zählt alle gültigen Knoten im Baum rekursiv (schließt NIL aus).	<code>RBTree.count_nodes()</code>
Aufgabe 6: Preorder-Traversierung (Datenwerte) Gibt eine Liste aller Values in der Reihenfolge: Wurzel -> L -> R aus.	<code>RBTree.get_values_preorder()</code>
Aufgabe 7: Postorder-Traversierung (Schlüsselwerte) Gibt eine Liste aller Keys in der Reihenfolge: L -> R -> Wurzel aus.	<code>RBTree.get_keys_postorder()</code>
Aufgabe 8: Anzahl aller echten Blätter zählen Zählt die Datenspeicherknoten, deren Kinder beide NIL-Objekte sind.	<code>RBTree.count_leaves()</code>
Aufgabe 9: Bestimmung der exakten Schwarzhöhe Folgt dem linken Pfad und ermittelt die korrekte RB-Baum-Schwarzhöhe.	<code>RBTree.get_black_height()</code>
Aufgabe 10: Den gesamten Baum leeren / zurücksetzen Setzt die Baumwurzel sicher und instantan auf einen leeren NIL-Knoten zurück.	<code>RBTree.clear()</code>
Aufgabe 11: Arithmetischer Durchschnitt aller Schlüssel Berechnet den exakten Mittelwert aller Keys im Baum als Fließkommazahl.	<code>RBTree.average_key()</code>
Aufgabe 12: Daten-Parsing aus einer JSON-Datei Liest Datensätze extern ein und speist sie direkt in das Baumobjekt ein.	Externe Funktion

2. Kompiliersichere Code-Snippets mit Erläuterungen

Aufgabe 1: Inorder-Traversierung

Erweitere RBTtree um eine Methode, die eine sortierte Liste aller Keys zurückgibt.

✓ Lösung

```
def inorder(self) -> list:
    def _inorder_rec(node, result):
        # HINWEIS: Wenn im Klausurcode NIL-Knoten anders abgefragt werden, .is_nil anpassen!
        if node and not node.is_nil:
            _inorder_rec(node.left, result)    # Zuerst links abtauchen (kleinere Werte)
            result.append(node.key)            # ACHTUNG: Hier .key oder .value nutzen, je nach
Aufgabe!                                     # Start an der Wurzel des Frameworks
            _inorder_rec(node.right, result)   # Danach rechts abtauchen (groessere Werte)
    res = []
    _inorder_rec(self._root, res)
    return res
```

Aufgabe 2: Iterative Elementsuche

Suche nach einem Schlüssel und gib den Datenwert oder None zurück.

✓ Lösung

```
def search(self, key: int) -> any:
    current = self._root                # Start an der obersten Baumwurzel
    while current and not current.is_nil: # Schleife laeuft, bis ein NIL-Blatt erreicht wird
        if key == current.key:
            return current.value         # Gefunden! Gibt das zugehoerige Datenfeld zurueck
        elif key < current.key:
            current = current.left       # Gesuchter Wert ist kleiner -> Gehe nach links
        else:
            current = current.right      # Gesuchter Wert ist groesser -> Gehe nach rechts
    return None                         # Schlüssel existiert im Baum nicht
```

Aufgabe 3: Globaler Maximum-Schlüssel

Gib den größten Schlüsselwert des Baums zurück. Wirf einen ValueError bei Leerheit.

✓ Lösung

```
def get_max_key(self) -> int:
    if self._root.is_nil:
        raise ValueError("Der Baum ist leer.") # Fehler werfen falls Wurzel leer ist
    current = self._root
    # HINWEIS: In binaeren Suchbaeumen liegen die groessten Werte IMMER ganz rechts aussen!
    while current.right and not current.right.is_nil:
        current = current.right                # Folge stur dem rechten Pfad
    return current.key                         # Letzter gueltiger Knoten ist das Maximum
```

Aufgabe 4: Globaler Minimum-Schlüssel

Gib den kleinsten Schlüsselwert des Baums zurück. Wirf einen ValueError bei Leerheit.

✓ Lösung

```
def get_min_key(self) -> int:
    if self._root.is_nil:
        raise ValueError("Der Baum ist leer.") # Fehler werfen falls Wurzel leer ist
    current = self._root
    # HINWEIS: In binären Suchbäumen liegen die kleinsten Werte IMMER ganz links aussen!
    while current.left and not current.left.is_nil:
        current = current.left # Folge stur dem linken Pfad
    return current.key # Letzter gültiger Knoten ist das Minimum
```

Aufgabe 5: Anzahl aller echten Knoten

Ermittle die Gesamtanzahl aller gültigen Datenspeicherknoten im Baum.

✓ Lösung

```
def count_nodes(self) -> int:
    # Hilfsfunktion zählt Wurzel + linken Unterbaum + rechten Unterbaum
    def _count_rec(node):
        if not node or node.is_nil: # Abbruchbedingung: NIL-Knoten zählen 0
            return 0
        return 1 + _count_rec(node.left) + _count_rec(node.right)
    return _count_rec(self._root)
```

Aufgabe 6: Preorder-Traversierung

Sammelt alle Datenwerte (Values) in der Reihenfolge: Wurzel -> Links -> Rechts.

✓ Lösung

```
def get_values_preorder(self) -> list:
    def _preorder_rec(node, result):
        if node and not node.is_nil:
            result.append(node.value) # KLAUSUR-TIPP: Erst verarbeiten (Wurzel), dann
            abtauchen!
            _preorder_rec(node.left, result)
            _preorder_rec(node.right, result)
    res = []
    _preorder_rec(self._root, res)
    return res
```

Aufgabe 7: Postorder-Traversierung

Sammelt alle Schlüssel (Keys) in der Reihenfolge: Links -> Rechts -> Wurzel.

✓ Lösung

```
def get_keys_postorder(self) -> list:
    def _postorder_rec(node, result):
        if node and not node.is_nil:
            _postorder_rec(node.left, result)    # Erst linken Ast verarbeiten
            _postorder_rec(node.right, result)   # Dann rechten Ast verarbeiten
            result.append(node.key)              # KLAUSUR-TIPP: Am Ende erst den aktuellen
    Knoten anfüegen!
    res = []
    _postorder_rec(self._root, res)
    return res
```

📋 Aufgabe 8: Anzahl aller echten Blätter zählen

Zähle alle Datenspeicherknoten, deren linker und rechter Nachfolger NIL-Objekte sind.

✓ Lösung

```
def count_leaves(self) -> int:
    def _count_rec(node) -> int:
        if not node or node.is_nil:
            return 0                                # NIL-Knoten sind keine Blätter
        # Ein Knoten ist ein echtes Blatt, wenn BEIDE Kinder NIL-Objekte sind
        if (not node.left or node.left.is_nil) and (not node.right or node.right.is_nil):
            return 1
        return _count_rec(node.left) + _count_rec(node.right)
    return _count_rec(self._root)
```

📋 Aufgabe 9: Bestimmung der exakten Schwarzhöhe

Ermittle die korrekte Schwarzhöhe des Baums durch Verfolgung des linken Pfads.

✓ Lösung

```
def get_black_height(self) -> int:
    height = 0
    current = self._root
    while current is not None:
        if current.is_black:                        # HINWEIS: .is_black prüft auf
            RBTTreeNode.Color.BLACK                # Erhöhe Zähler bei jedem schwarzen Knoten
            height += 1
        if current.is_nil:                          # Halt machen, sobald das NIL-Blatt mitgezählt
            wurde                                   # Stur nach links wandern reicht (Pfade sind
            current = current.left                  balanciert)
    return height
```

📋 Aufgabe 10: Den gesamten Baum leeren / zurücksetzen

Setze das Baumobjekt sicher in den initialen Leerzustand zurück.

✓ Lösung

```
def clear(self) -> None:
    # WICHTIG: Nutze die originale Initialisierungsmethode deines Frameworks!
    self._root = RBTreeNode.nil()
```

📋 Aufgabe 11: Arithmetischer Durchschnitt aller Schlüssel

Berechne das arithmetische Mittel aller Schlüsselwerte. Bei Leere wird 0.0 zurückgegeben.

✓ Lösung

```
def average_key(self) -> float:
    keys = self.inorder()                # Nutzt deine Methode aus Aufgabe 1
    (Schnittstelle!)
    if not keys:
        return 0.0                      # Verhindert Division durch Null
    (ZeroDivisionError)
    return sum(keys) / len(keys)        # Summe aller Schluesel geteilt durch die
    Elementanzahl
```

📋 Aufgabe 12: Daten-Parsing aus einer JSON-Datei

Eigenständige Funktion zum Parsen einer JSON-Datei und direktem Befüllen eines Baums.

✓ Lösung

```
from json import load                    # KLAUSUR-HINWEIS: Import MUSS ganz oben im Skript
stehen!

def load_tree_from_json(filepath: str) -> RBTree:
    tree = RBTree()                     # Erzeugt eine frische, leere Instanz des Baums
    # WICHTIG: Ersetze 'filepath' bzw. den String in open() durch den exakten Klausurnamen (z.B.
    "messung.json")
    with open(filepath, 'r') as f:
        data_list = load(f)             # Lädt JSON-Array direkt in Python-Datenstruktur
    for item in data_list:
        # ACHTUNG: Passe die JSON-Schlüssel ("key") exakt an das Format deines Professors an!
        tree.insert(item["key"], item["value"])
    return tree                          # Gibt den vollstaendig befuellten Baum zurueck
```